

Reliable and Trustworthy Artificial Intelligence

Assignment #4

G202648001 Tae-woo Kang

1. Exploration of α, β -CROWN Resources

α, β -CROWN ships pre-defined models in the `models/` directory, grouped by benchmark (toy, sdg, eran, oval, cifar10_resnet, marabou_cifar10, non_relu, and others), stored mainly as PyTorch weights (.pth, .pt, .model) with the Python files defining their architectures. The collection covers not only fully-connected networks but also convolutional ones such as ResNet — a broader range than Marabou's examples (ACAS Xu in .nnet and a few small MNIST .onnx models).

Verification setups are kept separately, as YAML files in `exp_configs/`, organised by competition year and feature; each lists the model, dataset, property (e.g. ϵ -ball robustness) and solver parameters in one file. This is the key difference from Marabou: Marabou is imperative — it loads a network through `marabou.py` and adds constraints in code — whereas α, β -CROWN is declarative, describing what to verify in a config, which makes it easy to define and run many instances as a batch.

2. Model and Dataset

To compare the two tools under identical conditions, I reused the model and inputs from the Marabou assignment (#3): a small fully-connected network $784 \rightarrow 32 \rightarrow 16 \rightarrow 10$ with ReLU activations, trained on MNIST to 92.0% test accuracy and stored in ONNX. The dataset is 10 MNIST images — one correctly classified input per digit (0–9) — with pixels normalised to $[0,1]$. Since the model and inputs are identical, any difference in verdict or runtime can be attributed to the verifier rather than the model.

3. Verification Property

For a reference input x correctly classified as digit c , local L_∞ robustness asks whether every x' with $\|x' - x\|_\infty \leq \epsilon$ (clipped to $[0,1]$ per pixel) is still classified as c . This is encoded as the negation of robustness — the unsafe region where some rival class j scores at least as high as the true class c . A 'safe' result proves that region is empty (robust); an 'unsafe' result returns a point in it (a counterexample). Verifying 10 digits at $\epsilon \in \{0.01, 0.05\}$ gives 20 verification instances.

4. Results

All 20 instances completed with no timeouts: 14 safe (robust) and 6 falsified (unsafe). At $\epsilon = 0.01$ every digit is robust, each proved by the linear-relaxation bound alone (without branching) in 0.33–0.56 s. The verdicts split at $\epsilon = 0.05$ (below).

Digit	Verdict	Verified by	Time (s)
0	unsafe	PGD attack	0.12
1	unsafe	PGD attack	0.15
2	robust	linear bound (incomplete)	0.37
3	robust	branch-and-bound (complete)	1.71
4	unsafe	PGD attack	0.13
5	robust	branch-and-bound (complete)	1.06
6	robust	linear bound (incomplete)	0.43
7	unsafe	PGD attack	0.12
8	unsafe	PGD attack	0.15
9	unsafe	PGD attack	0.13

Table 1. Verification results at $\epsilon = 0.05$ (mean over all 20 instances: 0.41 s; max: 1.71 s).

Two behaviours stood out. First, the escalation from incomplete to complete verification is visible: robust digits 2 and 6 needed only the linear bound, whereas digits 3 and 5 required full branch-and-bound and were the slowest cases (~ 1 – 1.7 s). Second, all six non-robust instances were caught by the PGD attack that runs before bound propagation, so their counterexamples appeared in ~ 0.12 s without any full verification. This division — cheap falsification by attack, cheap verification by bounds, and expensive branch-and-bound only for the borderline cases — is what makes α, β -CROWN efficient.

5. Comparison with Marabou

Marabou (#3) was run on digits 0, 1, 2 at the same two radii. On identical inputs and property the verdicts agree perfectly with α,β -CROWN, cross-validating both setups; the runtimes, however, diverge sharply.

Digit	ϵ	Verdict (both)	Marabou	α,β -CROWN
0	0.01	robust	0.44 s	0.56 s
0	0.05	not robust	45.85 s	0.12 s
1	0.01	robust	0.53 s	0.36 s
1	0.05	not robust	32.45 s	0.15 s
2	0.01	robust	0.34 s	0.42 s
2	0.05	robust	0.55 s	0.37 s

Table 2. Marabou vs α,β -CROWN on the same model and property.

Speed: at $\epsilon = 0.01$ the two tools are comparable (both under a second), but on the non-robust $\epsilon = 0.05$ cases the gap explodes — Marabou's SMT search took 30–46 s to return SAT, while α,β -CROWN falsified them ~ 250 – $380\times$ faster, since its gradient attack finds a counterexample before any complete search is needed. Scalability: α,β -CROWN combines linear relaxation with branch-and-bound and is built to scale to larger networks, whereas Marabou's SMT approach slows rapidly as model size and ϵ grow. Usability and supported features: α,β -CROWN supports ONNX and VNNLIB directly and has a built-in PGD attack, handling verification and falsification in one flow, while Marabou must solve a separate query per rival class.

6. α,β -CROWN: Strengths and Limitations

Strengths

- Much faster than Marabou here, with the largest gap on instances that have a counterexample.
- A built-in PGD attack finds counterexamples cheaply before any expensive verification.
- Sound yet efficient multi-level strategy: linear bound (incomplete) first, escalating to branch-and-bound (complete) only when needed.
- Supports ONNX and VNNLIB directly, making it easy to run many instances as a batch.

Limitations

- Here every $\epsilon = 0.01$ case was proved by the linear bound alone, so the complete (branch-and-bound) stage was barely exercised; a tighter ϵ or harder network is needed to really test it.
- As a bound-propagation method it can in principle return 'unknown' on harder networks, whereas Marabou's SMT search is exact (at a steep time cost).
- The tool is heavier than Marabou, presupposing a larger runtime environment (a deep-learning framework with GPU acceleration).